



INTERNSHIP PROJECT REPORT

Pearls AQI Predictor

An End-to-End Air Quality Index Forecasting System

Prepared by

Amna Altaf

Individual Internship Project
10P Shine Internship Program

Machine Learning | Data Engineering | MLOps | Environmental Forecasting

Abstract

Pearls AQI Predictor is an end-to-end machine-learning system developed to forecast Air Quality Index (AQI) for Pakistani cities 24, 48, and 72 hours ahead. The system combines historical and live environmental data from Open-Meteo and OpenWeather with feature engineering, XGBoost forecasting, Hopsworks feature and model management, FastAPI inference, Streamlit visualization, and GitHub Actions automation.

The final system uses temporal, weather, pollutant, lag, rolling, trend, and spatial features. Separate XGBoost models are trained for the three forecast horizons. The recorded results achieved R2 values of 0.6606, 0.6017, and 0.5865 for 24-, 48-, and 72-hour forecasts respectively. The project also addressed practical issues including noisy environmental data, missing values, API failures, feature leakage, training-inference inconsistencies, and Hopsworks availability. The resulting system provides a practical forecasting pipeline while demonstrating the limitations of longer-horizon AQI prediction.

Table of Contents

Section	Title
1. Introduction	
2. Problem Statement	
3. Project Scope	
4. Methodology	
5. System Architecture	
6. Data Collection and Processing	
7. AQI Calculation	
8. Feature Engineering	
9. Model Development and Training	
10. Model Evaluation	
11. Feature Store and Model Registry	
12. Live Inference and FastAPI Service	
13. Streamlit Dashboard	
14. Automation with GitHub Actions	
15. Challenges Faced	
16. Solutions and Improvements	
17. Limitations	
18. Final Project Outcome	
19. Future Work	
20. Conclusion	
21. References	
22. Appendices	

1. Introduction

Air quality changes continuously with pollutant concentrations, weather conditions, time, and location. Forecasting future AQI can provide advance information about expected pollution conditions and make environmental information more useful to end users.

Pearls AQI Predictor was developed as a practical forecasting system for Pakistani cities. Instead of focusing only on model training, the project combines data collection, preprocessing, feature engineering, live inference, API serving, dashboard visualization, model management, and automation into one workflow.

2. Problem Statement

AQI forecasting is difficult because pollution levels are noisy and affected by rapidly changing environmental conditions. Historical AQI alone is insufficient to describe future conditions, so weather, pollutant concentrations, temporal behaviour, and recent trends must also be considered.

The project addresses the problem of developing a practical system that uses historical and live environmental data to generate 24-, 48-, and 72-hour AQI forecasts while maintaining reliable feature generation and inference across multiple cities.

3. Project Scope

The final system covers nine Pakistani cities:

- Karachi
- Lahore
- Islamabad
- Rawalpindi
- Faisalabad
- Multan
- Quetta
- Hyderabad
- Sialkot

The configured historical period is 1 January 2024 to 30 July 2026, using an hourly time-series structure. The project covers the complete flow from data acquisition and feature engineering through model training, serving, visualization, and scheduled automation.

4. Methodology

The system follows an end-to-end workflow. Historical weather and air-quality data are obtained from Open-Meteo, while OpenWeather is used for live pollutant readings. Raw pollutant concentrations are converted into AQI using an internal EPA-style breakpoint implementation.

The data is cleaned, aligned by city and timestamp, and transformed into temporal, weather, pollutant, lag, rolling, trend, interaction, and spatial features. Hopsworks is used for feature and

model management when available, while local CSV files provide resilience. Separate XGBoost models are then trained for 24-, 48-, and 72-hour targets, evaluated with MAE, RMSE, and R2, and exposed through FastAPI and Streamlit.

5. System Architecture

The project separates data ingestion, feature engineering, feature storage, modelling, inference, visualization, and automation. The resulting architecture is designed so that an external service failure does not necessarily stop the entire pipeline.

Data Sources	Processing	Consumption
Open-Meteo	AQI calculation	FastAPI
OpenWeather	Preprocessing	Streamlit
Local CSV	Feature engineering	User forecasts
Hopsworks	XGBoost training	SHAP explanations
GitHub Actions	Model registration	Scheduled updates

Figure 1. High-level component flow of the Pearls AQI Predictor architecture.

6. Data Collection and Processing

6.1 Data Sources

Open-Meteo provides the historical weather and air-quality variables used by the project. These include temperature, relative humidity, surface pressure, wind speed and direction, cloud cover, precipitation, shortwave radiation, PM2.5, PM10, NO2, SO2, CO, O3, dust, and aerosol optical depth.

OpenWeather is used in the live pathway for current PM2.5, PM10, NO2, SO2, CO, and O3 observations. The different source schemas required explicit standardization before feature generation.

6.2 Local Data Persistence

- raw_all_cities.csv
- raw_all_cities_backup.csv
- features_all_cities.csv
- live_buffer.csv
- features_live_accumulated.csv

City-specific raw files are also retained. Local persistence is both a development convenience and a resilience mechanism for external service failures.

6.3 Cleaning and Alignment

- Convert timestamps to datetime and maintain Asia/Karachi time for live processing.
- Sort observations by city and time so temporal operations remain city-specific.

- Convert numeric fields using coercion so invalid values become missing values.
- Remove duplicate city-time observations while retaining the latest record.
- Drop rows with missing required values during model training rather than relying on extensive interpolation.

7. AQI Calculation

AQI is calculated internally rather than being taken directly from a single AQI field supplied by an external API. The implementation in `aqi_utils.py` uses EPA-style breakpoint logic for PM2.5 and PM10, calculates pollutant-specific sub-indices, and uses the maximum applicable sub-index as the final AQI.

Boundary handling keeps the calculated AQI within the range 0 to 500 and safely handles unavailable pollutant values. The same internal method is used for historical and live data to improve consistency across the system.

8. Feature Engineering

Feature engineering is a central part of the project because the raw environmental measurements do not directly represent all of the temporal and atmospheric information required for forecasting.

8.1 Temporal Features

- hour, day, month, weekday, weekend indicator, and season
- hour_sin, hour_cos, month_sin, month_cos, day_of_week_sin, and day_of_week_cos

8.2 Historical and Rolling Features

- AQI lags at 1h, 3h, 6h, 12h, and 24h, with additional longer lags in the advanced feature set.
- 3h, 6h, and 24h rolling mean, maximum, minimum, and standard deviation.
- AQI differences, change rate, and 24-hour trend features.

8.3 Environmental and Interaction Features

- PM ratio, wind-pollution interaction, and PM2.5-humidity interaction.
- Temperature and humidity differences.
- Wind components, EWMA, pressure trend, temperature inversion proxy, dispersion index, and stagnation index.
- Upstream smog drift and upstream AQI difference for cross-city spatial context.

A critical design decision was to calculate lag and rolling features within each city. This prevents one city's observations from being used as another city's historical context and eliminates a major source of cross-city feature leakage.

9. Model Development and Training

XGBoost was selected as the principal forecasting model for structured environmental data. Separate models are trained for target_aqi_24h, target_aqi_48h, and target_aqi_72h using the same engineered features and shared hyperparameter configuration.

Parameter	Value
n_estimators	3000
learning_rate	0.02
max_depth	7
subsample	0.8
colsample_bytree	0.8
tree_method	hist
early_stopping_rounds	50
eval_metric	RMSE
n_jobs	-1
random_state	42
enable_categorical	True

The final training configuration uses an 80/20 chronological split, with the earlier observations used for training and the later observations used for testing. A persistence baseline was also implemented, although its numerical results were not retained in the final artifacts. CatBoost was included in the project environment and experimentation, but corresponding saved comparison metrics were not available.

10. Model Evaluation

Model performance was assessed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R2. The final recorded XGBoost results are shown below.

Model	Horizon	MAE	RMSE	R2
XGBoost	24h	21.9895	34.3406	0.6606
XGBoost	48h	24.6796	37.1699	0.6017
XGBoost	72h	25.3176	37.9185	0.5865

The 24-hour model is the strongest of the three horizons. As the forecast horizon increases, MAE and RMSE increase while R2 decreases. This provides direct evidence that the model captures useful short-term structure but encounters greater uncertainty farther into the future.

11. Feature Store and Model Registry

Hopsworks was incorporated as the central feature and model management layer. The configured feature group is `aqi_features`, version 1. The feature group contains the environmental and engineered variables required by the forecasting pipeline.

The project also integrates Hopsworks Model Registry for model artifact management. Separate registry entries are created for the forecasting targets. Because Hopsworks was not consistently reliable in the final project state, local joblib model artifacts and CSV feature files were retained as fallbacks.

This fallback approach was particularly important when the feature store was unavailable, when feature insertion failed, or when free-tier limitations affected service availability.

12. Live Inference and FastAPI Service

The live inference process combines current OpenWeather pollutants, current weather information, and the recent local history maintained in `live_buffer.csv`. This history is necessary for recreating lag and rolling features required by the model.

The dedicated `inference_features.py` module constructs the serving-time feature row so that training and live prediction use equivalent feature semantics.

FastAPI endpoints

Endpoint	Purpose
<code>/health</code>	Service availability check
<code>/cities</code>	Configured city information
<code>/predict</code>	Forecast request
<code>/predict/{city}</code>	City-specific forecast

The service returns current AQI information together with 24-, 48-, and 72-hour forecasts. It includes error handling for unknown cities, missing model files, missing history, and failed live-data retrieval. The API is locally operational; public cloud deployment was not completed in the final repository state.

13. Streamlit Dashboard

The Streamlit application provides a user-facing interface organized into Home, Live AQI, Forecast, EDA, and Model Explainability sections.

- City-level AQI cards and city selection.
- A city map showing AQI conditions.
- Current AQI and temperature information.
- Forecast visualization for 24, 48, and 72 hours.
- Historical AQI trends.

- AQI alert classification.
- SHAP summary visualization for model explainability.

AQI alert categories

AQI Range	Dashboard Status
<= 50	Good
51-100	Moderate
101-150	Unhealthy for Sensitive Groups
151-200	Unhealthy - Orange Alert
201-300	Very Unhealthy - Red Alert
> 300	Hazardous

The dashboard can fall back to local data when the API is unavailable. It is locally operational but was not publicly deployed in the final project state.

14. Automation with GitHub Actions

Two workflows were implemented to automate recurring project operations.

Workflow	Schedule	Primary action
hourly_feature_pipeline.yml	0 * * * *	python fetch_live_data.py
daily_retrain.yml	0 0 * * *	python train.py

Both workflows support manual workflow_dispatch execution. Sensitive credentials are supplied through GitHub repository secrets. The automation structure was implemented, but the preserved artifacts do not provide conclusive evidence that the entire scheduled chain operated flawlessly over an extended period.

15. Challenges Faced

15.1 Cross-City Feature Leakage

A major modelling issue was the risk of calculating lag and rolling values across city boundaries. Global temporal sorting could cause one city's observation to become another city's lag value. The solution was to perform temporal operations within each city. This fixed a fundamental correctness problem in the feature pipeline.

15.2 Training-Inference Feature Mismatch

The training and live-serving paths were different code paths, creating a risk that the model would receive differently constructed features during prediction. A dedicated inference feature module was introduced to make serving-time feature generation consistent with training semantics.

15.3 Missing and Incomplete Data

Environmental observations can be incomplete or irregular. The project handled this through numeric coercion, missing-value preservation, duplicate removal, local history, and dropping incomplete rows during training.

15.4 External API Instability

Live and historical collection depends on external services that can experience timeouts, incomplete responses, or availability issues. Retry logic and graceful failure handling were introduced to reduce the impact of transient failures.

15.5 Hopsworks Reliability and Free-Tier Constraints

Hopsworks introduced availability, schema, insertion, and service-limit challenges. Local CSV fallbacks and best-effort external writes prevented the feature store from becoming a single point of failure.

15.6 Noisy AQI and Pollution Spikes

Sudden pollution spikes are difficult to predict. Additional lag, rolling, trend, weather, atmospheric, and spatial features were introduced to provide more context, but extreme events remained difficult.

15.7 Longer Forecast Horizons

The 48-hour and 72-hour forecasts were weaker than the 24-hour forecast. The system responded with richer temporal and environmental features, but uncertainty naturally remained higher at longer horizons.

15.8 Dependency and Automation Issues

The project kept application and pipeline dependencies in separate requirement files and used retry, fallback, and non-fatal external operations to improve reliability of automated workflows.

16. Solutions and Improvements

- City-aware lag and rolling calculations to prevent cross-city leakage.
- Dedicated inference feature generation to reduce training-serving skew.
- Richer temporal, atmospheric, interaction, and spatial features.
- Local CSV buffering and fallback for Hopsworks and live-service failures.
- Hourly feature ingestion and daily retraining workflows.
- Consistent internal AQI calculation across historical and live data.

17. Limitations

- AQI can change abruptly during unusual pollution or weather events.
- Prediction performance decreases for longer horizons.

- Public API data may contain gaps, inconsistent fields, and temporary outages.
- Different cities exhibit different environmental behaviour.
- Feature drift may occur as environmental distributions change over time.
- Hopsworks availability was not consistently stable.
- Public cloud deployment of the FastAPI and Streamlit applications was not completed.
- The final repository does not preserve formal EDA notebooks or exact SHAP feature rankings.
- Saved artifacts do not conclusively prove a single uninterrupted end-to-end automated cloud run.

18. Final Project Outcome

Pearls AQI Predictor was substantially completed with limitations. The final repository contains the major components required for an end-to-end forecasting system: historical and live data acquisition, AQI calculation, preprocessing, feature engineering, Hopsworks integration, XGBoost forecasting, evaluation, SHAP explainability, model registry integration, FastAPI serving, Streamlit visualization, GitHub Actions automation, and local resilience mechanisms.

Component	Status	Final note
Data ingestion	Implemented	Historical and live paths
Feature engineering	Implemented	Temporal, environmental, spatial
XGBoost forecasting	Implemented	24h / 48h / 72h
Hopsworks	Implemented with fallback	Availability limitations
FastAPI	Locally operational	Public deployment not completed
Streamlit	Locally operational	Public deployment not completed
Automation	Implemented	Full reliability not conclusively demonstrated

19. Future Work

- Expand geographic coverage with additional reliable city data.
- Increase the size and consistency of the historical dataset.
- Add redundant environmental data sources.
- Evaluate LSTM, GRU, and temporal transformer models.
- Introduce prediction intervals and uncertainty estimation.
- Add data-drift and model-performance monitoring.
- Improve automated retraining triggers and alerting.
- Strengthen spatial-temporal modelling.
- Complete stable public cloud deployment.

20. Conclusion

Pearls AQI Predictor demonstrates the development of a practical end-to-end environmental forecasting system rather than a standalone machine-learning notebook. The system integrates

data acquisition, internal AQI calculation, preprocessing, feature engineering, XGBoost modelling, feature and model management, live inference, API serving, dashboard visualization, explainability, and workflow automation.

The final models achieved R^2 values of 0.6606, 0.6017, and 0.5865 for the 24-, 48-, and 72-hour horizons. The decrease in performance with longer horizons reflects the inherent uncertainty of AQI forecasting. At the same time, the project demonstrated meaningful engineering solutions to feature leakage, missing data, external-service instability, training-inference mismatch, and operational resilience.

The final outcome is best described as a substantially completed, production-oriented forecasting system with clear limitations and clear opportunities for further development.

21. References

1. Open-Meteo - Archive API Documentation.
2. Open-Meteo - Air Quality API Documentation.
3. OpenWeather - Air Pollution API Documentation.
4. Hopsworks - Feature Store Documentation.
5. Hopsworks - Model Registry Documentation.
6. XGBoost - Documentation.
7. SHAP - Documentation.
8. FastAPI - Documentation.
9. Streamlit - Documentation.
10. GitHub Actions - Documentation.
11. U.S. Environmental Protection Agency - AQI Methodology and Breakpoints.

22. Appendices

Appendix A - Final Project Structure

```
Pearls AQI Predictor/
|-- README.md
|-- requirements.txt
|-- requirements-pipeline.txt
|-- runtime.txt
|-- hourly_feature_pipeline.yml
|-- daily_retrain.yml
|-- data/
|   |-- raw_all_cities.csv
|   |-- raw_all_cities_backup.csv
|   |-- features_all_cities.csv
|   |-- live_buffer.csv
|   |-- features_live_accumulated.csv
|   `-- city-level raw files
|-- feature_pipeline/
|   |-- config.py
|   |-- aqi_utils.py
```

```

| |-- feature_engineering.py
| |-- fetch_live_data.py
| |-- inference_features.py
| |-- backfill_history.py
| |-- backfill_extend_earlier.py
| |-- upload_to_hopsworks.py
| |-- reseed_buffer_from_hopsworks.py
| `-- audit_data_quality.py
|-- training_pipeline/
|   `-- train.py
|-- models/
|   `-- metrics.json
|-- fastapi/
|   `-- main.py
|-- streamlit/
|   `-- app.py
`-- notebooks/

```

Appendix B - Forecast Targets

```

target_aqi_24h
target_aqi_48h
target_aqi_72h

```

Appendix C - Final Performance

Horizon	MAE	RMSE	R2
24h	21.9895	34.3406	0.6606
48h	24.6796	37.1699	0.6017
72h	25.3176	37.9185	0.5865

Appendix D - AQI Alert Categories

```

AQI <= 50    -> Good
AQI 51-100   -> Moderate
AQI 101-150  -> Unhealthy for Sensitive Groups
AQI 151-200  -> Unhealthy - Orange Alert
AQI 201-300  -> Very Unhealthy - Red Alert
AQI > 300    -> Hazardous

```